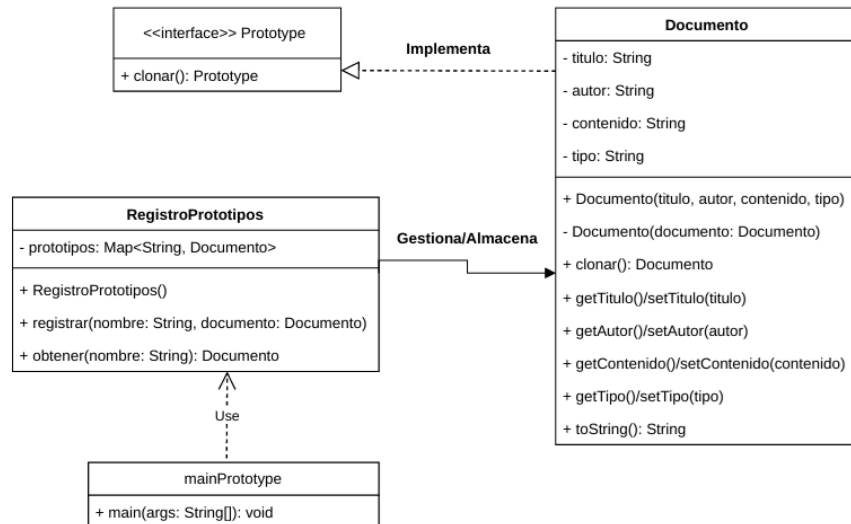


Patrón de Diseño: Prototype

1. Diagrama UML de clases
 - a. Diagrama UML Prototype



- b. Identificar y describir las clases, interfaces, atributos, métodos y relaciones involucradas.
 - **Prototype(interfaz):** Define el contrato básico que deben cumplir todos los objetos clonables a través del método `clonar()`.
 - **Documento:** Implementa la interfaz `Prototype`. Contiene los atributos de negocio (`titulo`, `autor`, `contenido`, `tipo`) y un constructor de copia privada que permite duplicar sus propiedades de manera segura y limpia.
 - **RegistroPrototipos:** Utiliza una estructura de datos `Map<String, Documento>` para almacenar prototipos preconfigurados y entregarlos ya clonados a los clientes cuando se soliciten por nombre.
 - **mainPrototype:** Pone en marcha el sistema, instancia un documento modelo, lo registra, solicita copias y demuestra la modificación independiente de los objetos clonados.

RELACIONES

- Documento implementa la interfaz Prototype.
- RegistroPrototipos agrega y gestiona objetos de tipo Documento.
- mainPrototype utiliza tanto el registro como los documentos para ejecutar la prueba.

2. Análisis del patrón

- a. **Problema:** Cuando la creación de un objeto requiere un proceso costoso o complejo (como realizar consultas pesadas, leer archivos grandes o inicializar numerosos atributos desde cero mediante el operador `new`), instanciarlo repetidamente afecta de manera negativa el rendimiento de la aplicación y genera un fuerte acoplamiento con clases concretas.
- b. **Solución:** El patrón Prototype permite copiar objetos ya existentes (prototipos) sin que el código dependa de sus clases concretas. En lugar de inicializar un nuevo objeto de forma manual, el sistema clona un modelo que ya se encuentra en memoria y operativo.
- c. **Consecuencias:**

- **Ventajas:** evita inicializaciones costosas y repetitivas; reduce drásticamente la cantidad de subclases de creación; y permite agregar o eliminar prototipos dinámicamente en tiempo de ejecución.
- **Desventaja:** clonar objetos muy complejos con referencias circulares o estructuras internas anidadas requiere implementar una lógica de copia profunda con sumo cuidado.

3. Implementación

- a. Código implementado
- b. Breve explicación del funcionamiento
 - La solución implementa el patrón de diseño Prototype, cuyo objetivo principal es permitir la creación de nuevos objetos mediante la clonación de instancias ya existentes, evitando procesos de inicialización costosos.
 - En este caso, la interfaz Prototype declara el método clonar(), el cual sirve como contrato común para todos los objetos que necesiten ser duplicados.
 - La clase Documento implementa dicha interfaz y define un constructor de copia privado (Documento(Documento documento)), permitiendo generar una nueva instancia en memoria copiando el estado exacto del objeto original de forma segura.
 - Para gestionar los prototipos de manera centralizada, se utiliza la clase RegistroPrototipos, la cual almacena los objetos en un diccionario (Map<String, Documento>) mediante un nombre identificador.
 - Cuando la clase cliente (mainPrototype) solicita un objeto mediante la instrucción: Documento informeEnero = registro.obtener("informe"); El registro busca el prototipo almacenado y retorna automáticamente una copia clonada e independiente del mismo.
 - Posteriormente, el cliente puede modificar el clon sin alterar el modelo original: informeEnero.setTitulo("Informe de Proyecto - Enero");
 - Esto permite obtener objetos idénticos o variaciones de manera instantánea, optimizando los recursos y facilitando la gestión dinámica de elementos repetitivos en la aplicación.

4. Principios del patrón Prototype

- **Principio de Responsabilidad Única (SRP):** La clase Documento encapsula tanto sus datos como su propia lógica de clonación, mientras que RegistroPrototipos se encarga exclusivamente de la administración y búsqueda de los prototipos almacenados.
- **Principio Abierto/Cerrado (OCP):** El código cliente opera interactuando a través de la abstracción o interfaces generales, permitiendo incorporar nuevos tipos de prototipos clonables sin necesidad de modificar el código existente del registro ni del cliente.
- **Principio de Inversión de Dependencias (DIP):** El sistema favorece el uso de interfaces como Prototype, desacoplando la lógica de creación de las implementaciones concretas.